CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 1

News

News

- Homework 02 posted
  - Due Feb 19 @ 11:59 pm EST on Gradescope.
  - LaTeX template available.

- Quiz: next week, HW02, Lecture 03 and 04.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 2

Finishing Quicksort

mergesort

- We saw `mergesort`.
- **Divide**: split list directly down the middle
- **Conquer**: sort each half
- **Combine**: merge sorted halves together

`merge` does all the work

- In `mergesort`, we are lazy when we divide.
- So we have to work to combine.

`[4,1,3,2]` → `[4,1]`,`[3,2]` → `[4,3]`,`[2,3]` → `[1,2,3,4]`

What if?

- Suppose we divide so that everything in `left` is smaller than everything in `right`:
- After sorting, no need for `merge`.
- `[5,1,3,8,6,2]` → `[1,3,2]`,`[5,8,6]`

What if?

- Suppose we divide so that everything in `left` is smaller than everything in `right`:
- After sorting, no need for `merge`.
- `[5,1,3,8,6,2]` → `[1,3,2]`,`[5,8,6]`
- This is what `partition` does!

Quicksort

```
def quicksort(arr, start, stop):
    """Sort arr[start:stop] in-place."""
    if stop - start > 1:
        pivot_ix = random.randrange(start, stop)
        pivot_ix = partition(arr, start, stop, pivot_ix)
        quicksort(arr, start, pivot_ix)
        quicksort(arr, pivot_ix+1, stop)
```

Time Complexity

- Average case: $\Theta(n \log n)$
- Worst case: $\Theta(n^2)$.
- Like with `quickselect`, worst case is **very rare**.

Mergesort vs Quicksort

- Mergesort has better worst case complexity.
- But in practice, Quicksort is often faster.
- Takes less memory, too.

Memory Requirements

- `merges` requires output array, $\Theta(n)$ additional space.
- `partition` works in-place, requires no additional space[1]
- Example: sorting 3 GB of data with 4 GB of RAM.

[1] Call stack for `quicksort` requires $\Theta(\log n)$ additional space.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 3

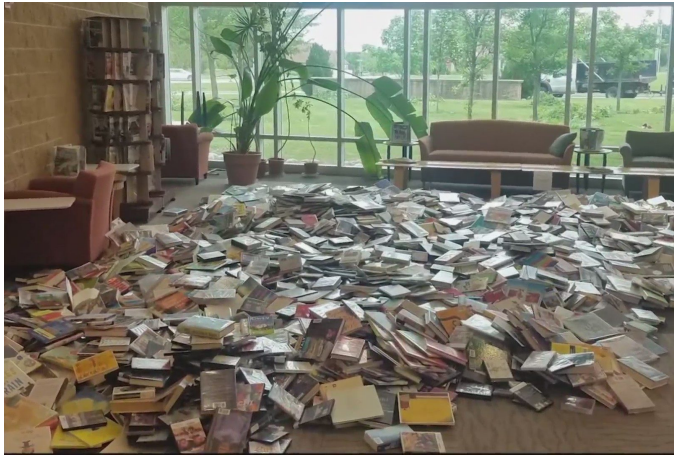
Data Structures and Dynamic Sets

Bookkeeping

- How do you store your books?

Bookkeeping

- How do you store your books?



Bookkeeping

- How do you store your books?

https://bookriot.com/how-to-organize-bookshelves/

Bookkeeping: Tradeoffs

- Messy:
  - No upfront cost.
  - Cost to search is high.
- Organized
  - Big upfront cost.
  - Cost to search is low.
- “Right” choice depends on how often we search.

Data Structures and Algorithms

- **Data structures** are ways of organizing data to make certain operations faster.
- Come with an upfront cost (preprocessing).
- “Right” choice of data structure depends on what operations we’ll be doing in the future.

Abstract vs. Concrete

- An **abstract data type** (ADT) is a formal description of a type's **interface**.
- A **data structure** is a concrete strategy for implementing an abstract data type.
  - Describes how data is stored in memory.
  - How to access the data.

Example: Stacks

- A **stack** is an ADT which supports two operations:
  - `push`: put a new object on to the "top"
  - `pop`: remove and return item at the "top"
- Often implemented using **linked lists**.
- But can also be implement with **(dynamic) arrays**.

Main Idea

A given abstract data type can be implemented in several ways, but some data structures are more natural choices than others.

Queries: Easy to Hard

- We've been thinking about **queries**.
  - Given a collection of data, is *x* in the collection?
- Querying is a fundamental operation.
  - Useful in a data science sense.
  - But also frequently performed in algorithms.
- There are several situations to think about.

Situation #1: Static Set, One Query

- **Given**: an unsorted collection of n numbers (or strings, etc.).
- In future, you will be asked single query.
- Which is better: linear search or sort + binary search?
  - Linear search: $\Theta(n)$ worst case.
  - Binary search would require sorting first in $\Theta(n \log n)$ worst case

Situation #2: Static Set, Many Queries

- **Given**: an unsorted collection of *n* numbers (or strings, etc.).
- In future, you will be asked **many** queries.
- Which is better: linear search or sort + binary search?
  - Depends on number of queries!

Exercise

Suppose you have a static set of n items. How long will it take[a] to perform k queries in total with:

1. linear search?
2. sort + binary search?

If $k = n/10$, which should you use?
What if $k = \log n$?

[a] On average. Assume the best case is rare.

Situation #3: Dynamic Set, Many Queries

- **Given**: a collection of *n* numbers (or strings, etc.).
- In future, you will be asked **many** queries *and* to **insert** or **remove** new elements.
- Best approach: ?

Binary Search?

- Can we still use binary search?
- **Problem**: To us binary search, we must maintain array in sorted order as we insert new elements.
- Inserting into array takes $\Theta(n)$ time in worst case.
  - Must “make room” for new element.
  - Can we use linked list with binary search?

Exercise

Suppose we have a collection of n elements. We make $n/4$ insertions and $n/4$ queries. How long will this take in total with

1. append to linked list + linear search?
2. maintain sorted array + binary search?

Today

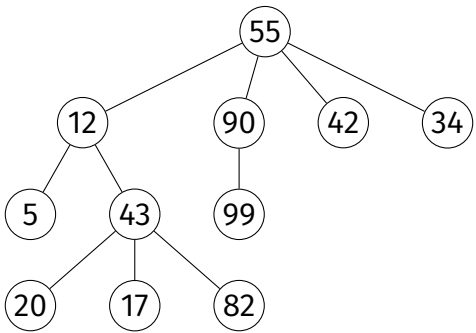
- Introduce (or review) **binary search trees**.
- BSTs support fast queries *and* insertions/deletion.
- Preserve sorted order of data after insertion/deletion.
- Can be modified to solve many problems efficiently.
  - Example: finding order statistics.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 4

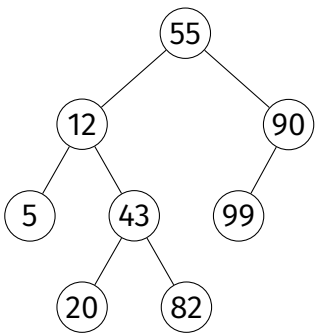
Binary Search Trees

Trees



Binary Trees

- Each node has *at most* two children (left and right).



Binary Search Tree

- A **binary search tree** (BST) is a binary tree that satisfies the following for any node x:
- if y is in *x*'s **left** subtree:

$$y.key \leq x.key$$

- if y is in *x*'s **right** subtree:

$$y.key \geq x.key$$

Assumption (for simplicity)

- We'll assume keys are unique (no duplicates).
- if y is in *x*'s **left** subtree:

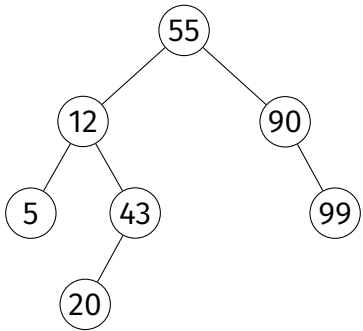
```
y.key < x.key
```

- if y is in *x*'s **right** subtree:

```
y.key > x.key
```

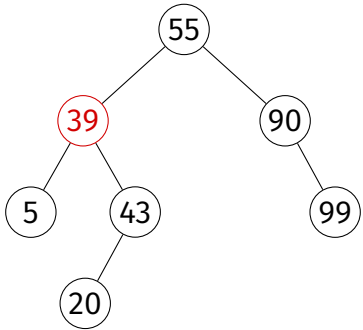
Example

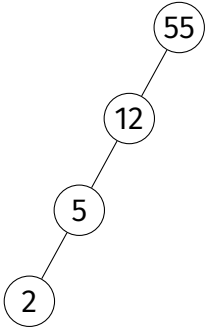
- This **is** a BST.



Example

- This is **not** a BST.





Exercise

Is this is a BST?

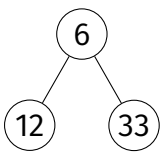
Height

- The **height** of a tree is the number of edges from the root to any leaf.
- Suppose a binary tree has n nodes.
- The **tallest** it can be is $\approx n$
- The **shortest** it can be is $\approx \log_2 n$

In Python

```
class Node:
    def __init__(self, key, parent=None):
        self.key = key
        self.parent = parent
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self, root: Node):
        self.root = root
```

In Python




```
root = Node(6)
n1 = Node(12, parent=root)
root.left = n1
n2 = Node(33, parent=root)
root.right = n2
tree = BinarySearchTree(root)
```

CS-GY 6033
Design and Analysis of Algorithms I

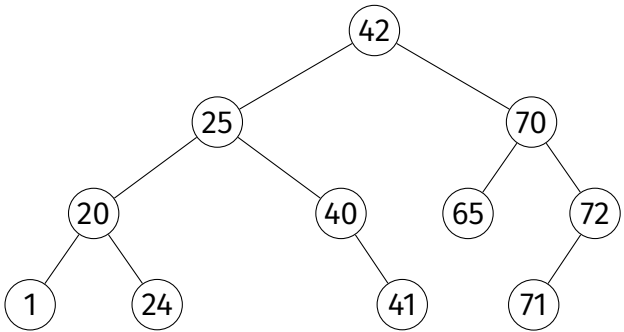
Lecture 4 | Part 5

Queries and Insertions in BSTs

Operations on BSTs

- We will want to:
  - **traverse** the nodes in sorted order by key
  - **query** a key (is it in the tree?)
  - **insert** a new key
  - **delete** an existing key

Inorder Traversal



Exercise

Implement `inorder` recursively so that it prints the keys of the nodes in the tree in sorted order.

```
def inorder(node):
    if node is not None:
        inorder(node.left)
        print(node.key)
        inorder(node.right)
```

Inorder Traversal

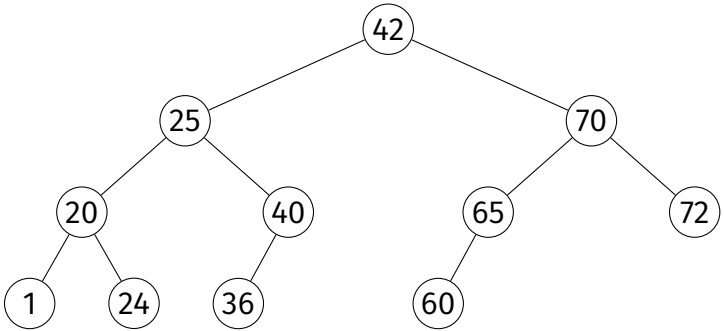
- Prints nodes in sorted order.
- Visits each node once, $\Theta(1)$ time in the call.
- Takes $\Theta(n)$ time.

Queries

- **Given**: a BST and a target, *t*.
- **Return**: `True` or `False`, is the target in the collection?

Queries

- Is 36 in the tree? 65? 23?



Queries

- Start walking from root.

- If current node is:
  - equal to target, return `True`;
  - too large (> target), follow left edge;
  - too small (< target), follow right edge;
  - `None`, return `False`

Queries, in Python

```
def query(self, target):
    """As method of BinarySearchTree."""
    current_node = self.root
    while current_node is not None:
        if current_node.key == target:
            return current_node
        elif current_node.key < target:
            current_node = current_node.right
        else:
            current_node = current_node.left
    return None
```

Exercise

Complete the recursive version of `query`.

```
def query_recursive(node, target):
    """As a 'free function'."""
    if node is None:
        return False

    if node.key == target:
        ...
    elif ...:
        ...
    else:
        ...
```

Queries (Recursive)

```
def query_recursive(node, target):
    """As a 'free function'."""
    if node is None:
        return False

    if node.key == target:
        return node
    elif node.key < target:
        return query_recursive(node.right, target)
    else:
        return query_recursive(node.left, target)
```

Queries, Analyzed

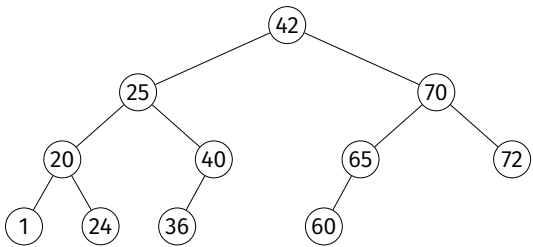
- Best case: $\Theta(1)$.
- Worst case: $\Theta(h)$, where h is **height** of tree.

Insertion

- **Given**: a BST and a new key, *k*.
- **Modify**: the BST, inserting *k*.
- Must **maintain** the BST properties.

Insertion

- Insert 23 into the BST.



Insertion (The Idea)

- Traverse the tree as in `query` to find empty spot where new key should go, keeping track of last node seen.
- Create new node; make last node seen the parent, update parent's children.
- Be careful about inserting into empty tree!

```
def insert(self, new_key):
    # assume new_key is unique
    current_node = self.root
    parent = None

    # find place to insert the new node
    while current_node is not None:
        parent = current_node
        if current_node.key < new_key:
            current_node = current_node.right
        else: # current_node.key > new_key
            current_node = current_node.left

    # create the new node
    new_node = Node(key=new_key, parent=parent)

    # if parent is None, this is the root. Otherwise, update the
    # parent's left or right child as appropriate
    if parent is None:
        self.root = new_node
    elif parent.key < new_key:
        parent.right = new_node
    else:
        parent.left = new_node
```

Insertion, Analyzed

- Worst case: $\Theta(h)$, where h is **height** of tree.

Main Idea

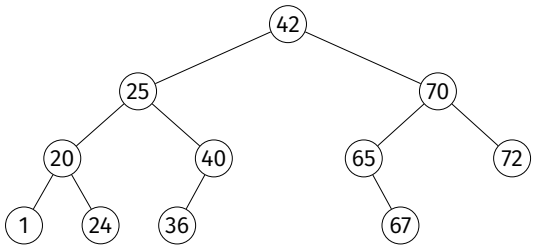
Querying and insertion take $\Theta(h)$ time in the worst case, where h is the height of the tree.

Deletion

- **Given**: a key in the BST.
- **Modify**: the BST, deleting the key.
- Must **maintain** the BST properties.
- This is a little trickier.

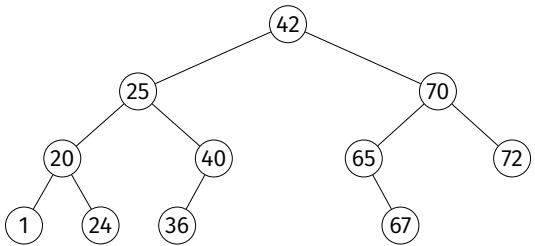
Deletion: Case 1 (Easy)

- Delete 36 from the BST.



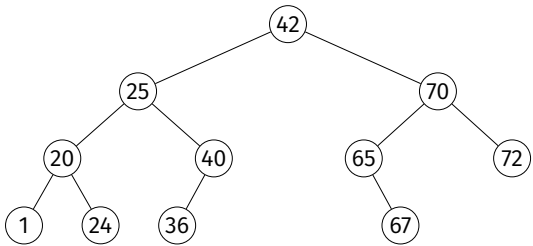
Deletion: Case 2 (Also Easy)

- **Exercise:** Delete 65 from the BST.



Deletion: Case 3 (Tricky)

- Delete 42 from the BST.

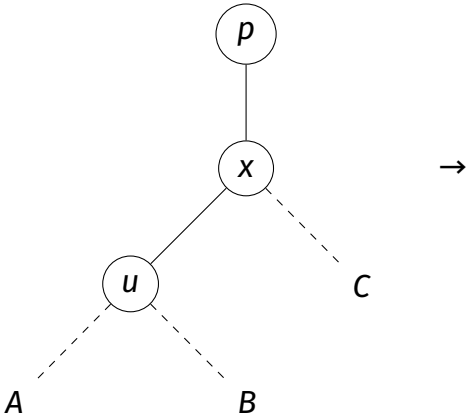


Deletion

- If node has no children (leaf), **easy**.
- If node has one child, also **easy**.
- Otherwise, a little trickier.
- Idea: **rotate**[2] node to bottom, preserving BST. When it is a leaf or has one child, delete it.

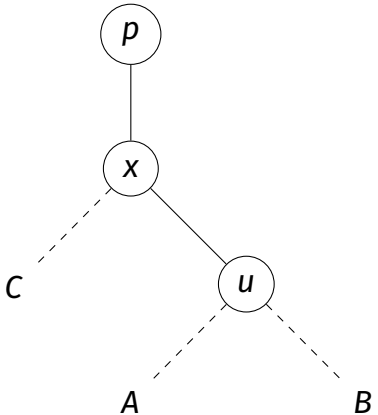
[2] Most books take a different approach with the same time complexity.

(Right) Rotation



→

(Left) Rotation



→

Claim

Left rotate and right rotate preserve the BST property.

```
def _right_rotate(self, x):
    u = x.left
    B = u.right
    C = x.right
    p = x.parent

    x.left = B
    if B is not None: B.parent = x

    u.right = x
    x.parent = u

    u.parent = p

    if p is None:
        self.root = u
    elif p.left is x:
        p.left = u
    else:
        p.right = u
```

Deletion Analyzed

- Each rotate takes $\Theta(1)$ time.
- $O(h)$ rotations until node becomes leaf.
- So $\Theta(h)$ time in the worst case.

Main Idea

Insertion, deletion, and querying all take $\Theta(h)$ time in the worst case, where h is the height of the tree.

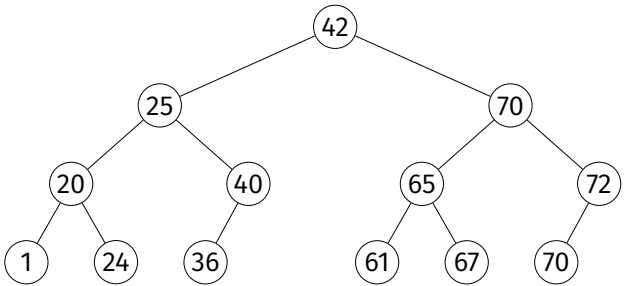
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 6

Balanced and Unbalanced BSTs

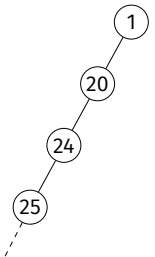
Binary Tree Height

- In case of very balanced tree, $h = \Theta(\log n)$.
  - Query, insertion take worst case $\Theta(\log n)$ time in a **balanced** tree.



Binary Tree Height

- In the case of very unbalanced tree, $h = \Theta(n)$.
  - Query, insertion take worst case $\Theta(n)$ time in **unbalanced** trees.



Unbalanced Trees

- Occurs if we insert items in (close to) sorted or reverse sorted order.

Example

- Insert 1, 2, 3, 4, 5, 6, 7, 8 (in that order).

Time Complexities

query	$\Theta(h)$
insertion	$\Theta(h)$

Where h is height, and $h = \Omega(\log n)$ and $h = O(n)$.

Time Complexities (Balanced)

query	$O(\log n)$
insertion	$O(\log n)$
deletion	$O(\log n)$

Where h is height, and $h = \Omega(\log n)$ and $h = O(n)$.

Worst Case Time Complexities (Unbalanced)

query	$\Theta(n)$
insertion	$\Theta(n)$
deletion	$\Theta(n)$

- The worst case is **bad**.
  - Worse than using a sorted array!
- The worst case is **not rare**.

Main Idea

The operations take linear time in the worst case **unless** we can somehow ensure that the tree is **balanced**.

Self-Balancing Trees

- There are variants of BSTs that are **self-balancing**.
  - Red-Black Trees, AVL Trees, etc.
- Quite complicated to implement correctly.
- But their height is **guaranteed** to be ~ $\log n$.
- So insertion, query take $\Theta(\log n)$ in worst case.

Warning!

If asked for the time complexity of a BST operation, be careful! A common mistake is to say that insertion/query are $\Theta(\log n)$ without being told that the tree is balanced.

Main Idea

In general, insertion/query take $\Theta(h)$ time in worst case. If tree is balanced, $h = \Theta(\log n)$, so they take $\Theta(\log n)$ time. If tree is badly unbalanced, $h = O(n)$, and they can take $O(n)$ time.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 7

Augmenting BSTs

Modifying BSTs

- Perhaps more than most other data structures, BSTs must be modified (**augmented**) to solve unique problems.

Order Statistics

- Given n numbers, the **kth order statistic** is the kth smallest number in the collection.

Dynamic Set, Many Order Statistics

- Quickselect finds any order statistic in linear expected time.
- This is efficient for a static set.
- Inefficient if set is dynamic.

Goal

- Create a **dynamic** set data structure that supports fast computation of **any** order statistic.

BST Solution

- For each node, keep attribute `.size`, containing # of nodes in subtree rooted at current node
- Property:[3] `x.size = x.left.size + x.right.size + 1`

[3] If a left or right child doesn't exist, consider its size zero.

Computing Sizes

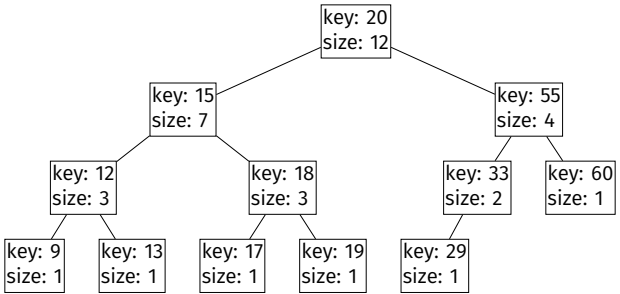
```
def add_sizes_to_tree(node):
    if node is None:
        return 0
    left_size = add_sizes_to_tree(node.left)
    right_size = add_sizes_to_tree(node.right)
    node.size = left_size + right_size + 1
    return node.size
```

Note

- Also need to maintain size upon inserting a node.

Computing Order Statistics

- 8th? 2nd? 12th



Augmenting Data Structures

- This is just one example, but many more.
- Understanding how BSTs work is key to augmenting them.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 8

Priority Queues

Priority Queues

- A **priority queue** is an abstract data type representing a collection.
- Each element has a **priority**.
- Supports operations[4]:
  - `.pop_highest_priority()`
  - `.insert(value, priority)`
  - `.is_empty()`

[4] and possibly more, like `.increase_priority`

Example

```
>>> er = PriorityQueue()
>>> er.insert('flu', priority=1)
>>> er.insert('heart attack', priority=20)
>>> er.insert('broken hand', priority=10)
>>> er.pop_highest_priority()
'heart attack'
>>> er.pop_highest_priority()
'broken hand'
```

Applications

- Scheduling.
- Useful in algorithms.
  - E.g., Prim's algorithm for Minimum Spanning Trees

Array Implementation

- We *can* implement a priority queue with a **(dynamic) array**.

- `.insert(k, p)`
  - append (value, priority) pair: $\Theta(1)$ amortized time

- `.pop_highest_priority()`
  - find entry with highest priority: $\Theta(n)$ time
  - remove it: $O(n)$ time

Exercise

What is the time needed for `.insert` and `.pop_highest_priority` if we maintain the array in **sorted order** of priority?

Array Implementation (Variant)

- Alternatively, maintain dynamic array in **sorted order** of priority.

- `.insert(k, p)`
  - find place in sorted order: $\Theta(\log n)$ time worst case
  - actually insert: $\Theta(n)$ time worst case

- `.pop_highest_priority()`
  - remove/return last entry: $\Theta(1)$ time

Main Idea

If we made no modifications, a sorted array would be great. But we want a data structure with quick remove/return even after being modified.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 9

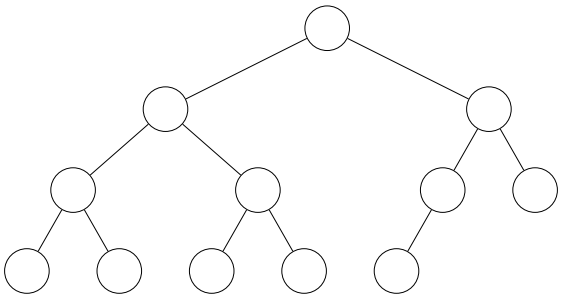
Binary Heaps

Binary Heaps

- A **binary heap** is a **binary tree** data structure often used to implement **priority queues**.

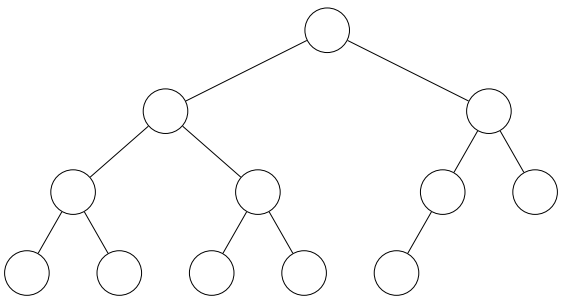
Complete Binary Trees

- A binary tree is **complete** if every level is filled, except for possibly the last (which fills from left to right).



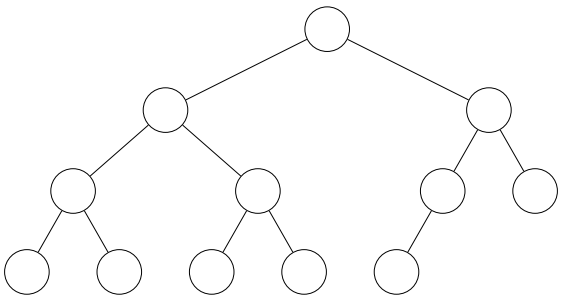
Node Height

- The **height** of node in a tree is the largest number of edges along any path to a leaf.
- The **height** of a tree is the height of the root.



Complete Tree Height

- The height of a complete binary tree with n nodes is $\Theta(\log n)$.

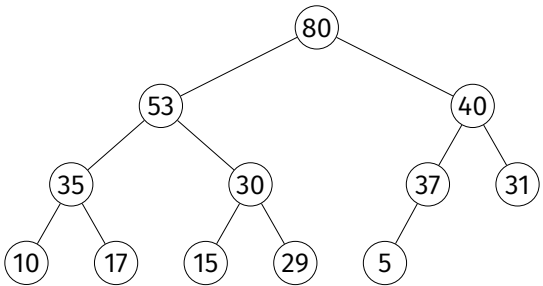


Binary Min Heap Properties

- A **binary min heap** is a binary tree with three additional properties:
  1. Each node has a **key**.
  2. **Shape**: the tree is complete.
  3. **Min-Heap**: the key of a node is ≤ the key of each of its children.

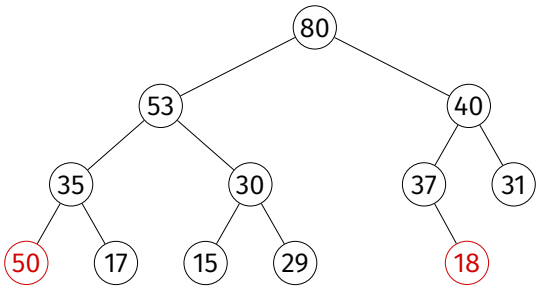
Example

- This is a binary max-heap.



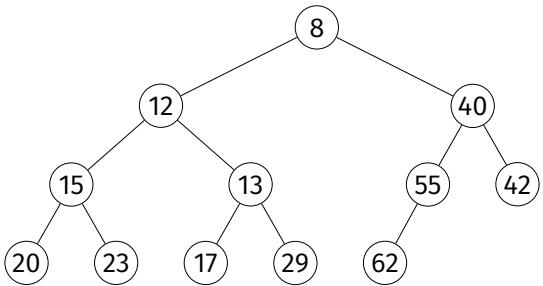
Example

- This is **not** a binary max-heap.



Example

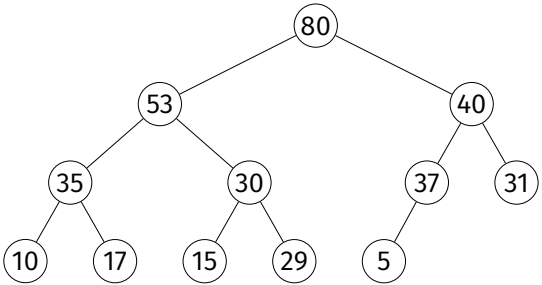
- This is a binary min-heap.



Representation

- One representation: nodes are **objects** with pointers to children.
- But due to completeness property, we can store a binary heap **compactly** in a (dynamic) array.

Array Representation



- `.left_child(i)`
- `.right_child(i)`
- `.parent(i)`

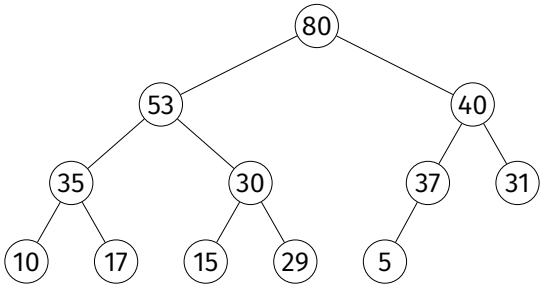
Exercise

Why would we prefer representing a binary heap with an array rather than as objects with pointers to children?

Operations

- `.max()`
  - Return (but do not remove) the max key
- `.increase_key(i, new_key)`
  - Increase key of node *i*, maintaining heap
- `.insert(key)`
  - Insert new node, maintaining heap
- `.pop_max()`
  - Remove max-key node, return key

.max



80	53	40	35	30	37	31	10	17	15	29	5
0	1	2	3	4	5	6	7	8	9	10	11

.max

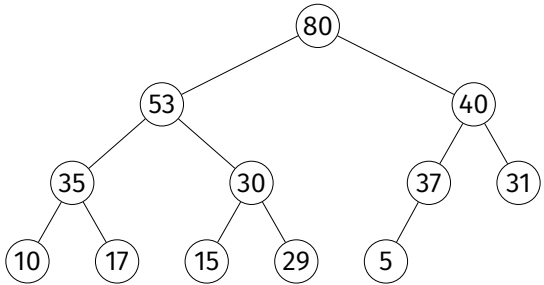
```
class MaxHeap:

    def __init__(self, keys=None):
        if keys is None:
            keys = []
        self.keys = keys

    def max(self):
        return self.keys[0]
```

`.max`

- Takes $\Theta(1)$ time.

.increase_key

```
.increase_key(9, key=60)
```



80	53	40	35	30	37	31	10	17	15	29	5
0	1	2	3	4	5	6	7	8	9	10	11

.increase_key

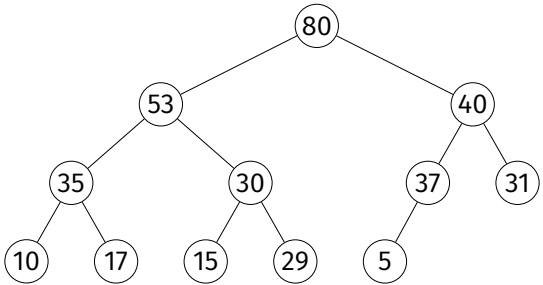
```
def increase_key(self, ix, key):
    if key < self.keys[ix]:
        raise ValueError('New key is smaller.')

    self.keys[ix] = key
    while (
            parent(ix) >= 0
            and
            self.keys[parent(ix)] < key
        ):
        self._swap(ix, parent(ix))
        ix = parent(ix)
```

`.increase_key`

- Takes $O(\log n)$ time.

.insert

.insert(key=60)



80	53	40	35	30	37	31	10	17	15	29	5
0	1	2	3	4	5	6	7	8	9	10	11

Exercise

Implement `.insert`.

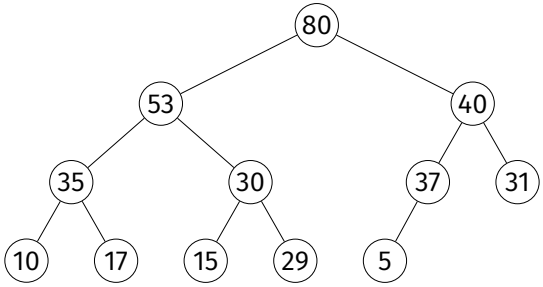
.insert

```
def insert(self, key):
    self.keys.append(key)
    self.increase_key(
        len(self.keys)-1, key
    )
```

`.insert`

- Takes $O(\log n)$ time (amortized).

.pop_max_key



80	53	40	35	30	37	31	10	17	15	29	5
0	1	2	3	4	5	6	7	8	9	10	11

.pop_max_key

```
def pop_max_key(self):
    if len(self.keys) == 0:
        raise IndexError('Heap is empty.')
    highest = self.max()
    self.keys[0] = self.keys[-1]
    self.keys.pop()
    self._push_down(0)
    return highest
```

`._push_down(i)`

- Assume that left and right subtrees of node *i* are max heaps, but key of *i* is possibly too small.

- Push it down until heap property satisfied.
  - Recursively swap with largest of left and right child.

._push_down()

```
def _push_down(self, i):
    left = left_child(i)
    right = right_child(i)
    if (
            left < len(self.keys)
            and
            self.keys[left] > self.keys[i]
    ):
        largest = left
    else:
        largest = i

    if (
        right < len(self.keys)
        and
        self.keys[right] > self.keys[largest]
    ):
        largest = right

    if largest != i:
        self._swap(i, largest)
        self._push_down(largest)
```

`.pop_max_key`

- `._push_down(i)` takes $O(h)$ where h is i's height
- Since $h = O(\log n)$, `.pop_max_key` takes $O(\log n)$ time.

Summary

For a binary heap[5]:

`.max`	$\Theta(1)$
`.increase_key`	$O(\log n)$
`.insert`	$O(\log n)$
`.pop_max_key`	$O(h) = O(\log n)$

[5] There are other heap data structures. Fibonacci heaps have $\Theta(1)$ insert and increase key, but slower for small n.

Implementing Priority Queues

- Can use max heaps to implement priority queues.
- But a priority queue has values *and* keys.

```
pq.insert('heart attack', priority=20)
```

Trick

- Heap keys need not be integers.
- Need only be comparable.
- Can store key and value with a `tuple`.

Tuple Comparison

- In Python, tuple comparison is lexicographical.
  - Compare first entry; if tie, compare second, etc.

```
>>> (10, 'test') > (5, 'zzz')
True
>>> (10, 'test') > (10, 'zzz')
False
```

Trick

- Use 2-tuples: priority in 1st spot, value in 2nd.

```
class PriorityQueue:

    def __init__(self):
        self._heap = MaxHeap()

    def insert(self, value, priority):
        self._heap.insert((priority, value))

    def pop_highest_priority(self):
        return self._heap.pop_max()

    def max(self):
        return self._heap.max()

    def is_empty(self):
        return not bool(self._heap.keys)
```

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 4 | Part 10

Example: Online Median

Online Median

- **Given**: a stream of numbers, one at a time.
- **Compute**: the median of all numbers seen so far.
- **Design**: a data structure with the following operations:
  - `.insert(number)`: in $\Theta(\log n)$ time
  - `.median()`: in $\Theta(1)$ time

Review

- Given an array, we can compute the median in:
  - $\Theta(n \log n)$ time by sorting
  - $\Theta(n)$ (expected) time with quickselect

- But modifying the array and repeating is costly.

Exercise

How could we use **two** heaps to store a collection of numbers so that the median is at the top of one of them?